



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

**SECP3623
Database Programming
2025/2026 SEMESTER 1
Assignment 1 (10%)**

Name	:	Ahmad Adib Zikri bin A.Mazlam
Matric Id	:	A23CS0205
Section	:	02

Name	:	Muhammad Afiq Danial bin Rozaidie
Matric Id	:	A23CS0117
Section	:	02

Lecturer: DR. SHAMINI A/P RAJA KUMARAN

TABLE OF CONTENTS

Task 1: Database Creation and Integrity Constraints	3
Task 2: Data Manipulation and Filtering	7
Task 3 – Reporting and Aggregation	15

Task 1: Database Creation and Integrity Constraints

1. CREATE a database and USE the database

```
CREATE DATABASE hostel_mgmt_adibafiq;  
use hostel_mgmt_adibafiq;
```

Output:

✓	36	14:30:15	CREATE DATABASE hostel_mgmt_adibafiq	1 row(s) affected
✓	37	14:30:17	use hostel_mgmt_adibafiq	0 row(s) affected

Explanation:

Create a database named hostel_mgmt_adibafiq and use its database throughout all the tasks.

2. CREATE TABLE for entity and referential integrity

```
CREATE TABLE room_types (  
    type_id INT PRIMARY KEY,  
    type_name VARCHAR(10),  
    rent DECIMAL(10,2),  
    deposit DECIMAL(10,2),  
    capacity INT  
);  
  
CREATE TABLE rooms (  
    room_id VARCHAR(5) PRIMARY KEY,  
    type_id INT,  
    room_no VARCHAR(10),  
    floor_no INT,  
    is_occupied BOOLEAN,  
    FOREIGN KEY (type_id) REFERENCES room_types(type_id)  
);
```

```
CREATE TABLE students (
    student_id VARCHAR(10) PRIMARY KEY,
    room_id VARCHAR(5),
    fname VARCHAR(30) NOT NULL,
    lname VARCHAR(30) NOT NULL,
    status VARCHAR(20),
    checkin_date DATE,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

```
CREATE TABLE maintenance (
    maint_id INT PRIMARY KEY,
    room_id VARCHAR(5),
    issue_desc TEXT,
    severity VARCHAR(15),
    status VARCHAR(10),
    reported_on DATE,
    resolved_on DATE,
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);
```

```
CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
    student_id VARCHAR(10),
    amount DECIMAL(10,2),
    paid_on DATE,
    method VARCHAR(10),
    note TEXT,
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

Output:

✓	42	14:48:06	CREATE TABLE room_types (type_id INT PRIMARY KEY, type_nam...	0 row(s) affected
✓	43	14:48:06	CREATE TABLE rooms (room_id VARCHAR(5) PRIMARY KEY, type_j...	0 row(s) affected
✓	44	14:48:06	CREATE TABLE students (student_id VARCHAR(10) PRIMARY KEY, ...	0 row(s) affected
✓	45	14:48:06	CREATE TABLE maintenance (maint_id INT PRIMARY KEY, room_id...	0 row(s) affected
✓	46	14:48:06	CREATE TABLE payments (payment_id INT PRIMARY KEY, student...	0 row(s) affected

Explanation:

For the first table, create a table named *room_types* that stores information about different room categories. It includes a primary key (*type_id*), the room type name, rental price, deposit amount, and the room capacity. For the datatypes that have been used in this table are int, varchar and decimal.

For the next table, create rooms with a name, *rooms* that contain the information about the room, which floor it is and the room vacancy. This table has primary key (*room_id*), room number, floor number, the occupancy status and room type id that references table *room_types*. int, varchar and boolean are the datatypes that have been used in this table.

Next, create a *students* table to store the information and details for each student who is assigned for each room. This table contains primary key (*student_id*), their first name along with last name, their activeness status, hostel check-in date and room id that references the table *rooms*. The datatypes used in this table include varchar and date. NOT NULL constraint is used only for both first name and last name.

The next table is the *maintenance* table which stores information about a room that has an issue with its own severity that comes from students' reports. The list of columns for this table are maintenance id, issue description, issue severity, issue's status, date of report, date of resolve issue and room id that reference from table *rooms*. The datatypes used in this table include int, varchar, text and date.

The last table creates a *payments* table to store payment records made by students. It includes the payment ID as the primary key, the student's ID, the amount paid, date of payment, payment method, and any additional notes. It also links each payment to a specific student using a foreign key reference. The datatypes like int, decimal, varchar, text and date were used for the columns in this table.

3. ALTER TABLE students to add an email column to students and mark it UNIQUE

```
ALTER TABLE students  
  ADD COLUMN email VARCHAR(30) UNIQUE;
```

Output:

✓	47	14:50:22	ALTER TABLE students ADD COLUMN email VARCHAR(30) UNIQUE	0 row(s) affected
---	----	----------	--	-------------------

4. CREATE TABLE test_table and DROP the table

```
CREATE TABLE test_table (  
  test_id INT PRIMARY KEY  
);
```

```
DROP TABLE test_table;
```

Output:

✓	48	14:57:31	CREATE TABLE test_table (test_id INT PRIMARY KEY)	0 row(s) affected
✓	49	14:57:37	DROP TABLE test_table	0 row(s) affected

Task 2: Data Manipulation and Filtering

I. INSERT INTO the data into all tables that created

Table room_types have 10 unique rows with the fixed type name, rent, deposit and capacity.

```
# Data Insertion (DML)
INSERT INTO room_types (type_id, type_name, rent, deposit, capacity)
VALUES
(1, 'Single', 550.00, 250.00, 1),
(2, 'Double', 450.00, 150.00, 2),
(3, 'Family', 750.00, 350.00, 4),
(4, 'Premium', 950.00, 450.00, 2),
(5, 'Single', 530.00, 200.00, 1),
(6, 'Double', 420.00, 120.00, 2),
(7, 'Family', 820.00, 300.00, 5),
(8, 'Premium', 980.00, 500.00, 2),
(9, 'Single', 580.00, 220.00, 1),
(10, 'Double', 460.00, 180.00, 2);

SELECT * FROM room_types;
```

Output:

	type_id	type_name	rent	deposit	capacity
▶	1	Single	550.00	250.00	1
	2	Double	450.00	150.00	2
	3	Family	750.00	350.00	4
	4	Premium	950.00	450.00	2
	5	Single	530.00	200.00	1
	6	Double	420.00	120.00	2
	7	Family	820.00	300.00	5
	8	Premium	980.00	500.00	2
	9	Single	580.00	220.00	1
	10	Double	460.00	180.00	2
*	NULL	NULL	NULL	NULL	NULL

✓	50	15:04:28	INSERT INTO room_types (type_id, type_name, rent, deposit, capacity) ...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
✓	51	15:04:28	SELECT * FROM room_types LIMIT 0, 50000	10 row(s) returned

II. INSERT INTO TABLE rooms

Table rooms have 10 rows with the uniquely assigned id and number. Each room has its own types.

```
INSERT INTO rooms (room_id, room_no, floor_no, is_occupied, type_id)
VALUES
('R001', 'A101', 1, TRUE, 1),
('R002', 'A102', 1, TRUE, 2),
('R003', 'A103', 1, FALSE, 3),
('R004', 'B201', 2, TRUE, 4),
('R005', 'B202', 2, FALSE, 5),
('R006', 'B203', 3, TRUE, 6),
('R007', 'C301', 3, TRUE, 7),
('R008', 'C302', 4, FALSE, 8),
('R009', 'C303', 4, TRUE, 9),
('R010', 'C304', 4, FALSE, 10);
```

```
SELECT * FROM rooms;
```

Output:

```
✓ 53 15:09:59 INSERT INTO rooms (room_id, room_no, floor_no, is_occupied, type_id) ... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
✓ 54 15:09:59 SELECT * FROM rooms LIMIT 0, 50000 10 row(s) returned
```

	room_id	type_id	room_no	floor_no	is_occupied
▶	R001	1	A101	1	1
	R002	2	A102	1	1
	R003	3	A103	1	0
	R004	4	B201	2	1
	R005	5	B202	2	0
	R006	6	B203	3	1
	R007	7	C301	3	1
	R008	8	C302	4	0
	R009	9	C303	4	1
	R010	10	C304	4	0
*	NULL	NULL	NULL	NULL	NULL

IV. INSERT INTO TABLE maintenance

The maintenance table contains a total of 15 records detailing various issues reported in different rooms, their severity, status, and dates of reporting and resolving.

```
INSERT INTO maintenance (maint_id, room_id, issue_desc, severity, status, reported_on, resolved_on)
VALUES
(1, 'R001', 'Aircond not cooling', 'HIGH', 'RESOLVED', '2025-01-05', '2025-01-07'),
(2, 'R002', 'Broken desk', 'LOW', 'RESOLVED', '2025-01-10', '2025-01-12'),
(3, 'R003', 'Water leak', 'MEDIUM', 'OPEN', '2025-02-03', NULL),
(4, 'R004', 'Power socket malfunction', 'MEDIUM', 'RESOLVED', '2025-02-08', '2025-02-09'),
(5, 'R005', 'Window crack', 'LOW', 'OPEN', '2025-03-02', NULL),
(6, 'R006', 'Internet not working', 'HIGH', 'OPEN', '2025-03-15', NULL),
(7, 'R007', 'Light bulb flickering', 'LOW', 'RESOLVED', '2025-04-05', '2025-04-30'),
(8, 'R008', 'Door hinge broken', 'MEDIUM', 'OPEN', '2025-05-11', NULL),
(9, 'R009', 'Aircond noise', 'LOW', 'RESOLVED', '2025-05-20', '2025-06-11'),
(10, 'R010', 'Floor tiles loose', 'MEDIUM', 'OPEN', '2025-06-01', NULL),
(11, 'R003', 'Ceiling paint peeling', 'LOW', 'OPEN', '2025-06-20', NULL),
(12, 'R005', 'Desk drawer stuck', 'LOW', 'RESOLVED', '2025-06-18', '2025-06-19'),
(13, 'R002', 'Aircond water dripping', 'MEDIUM', 'OPEN', '2025-07-01', NULL),
(14, 'R008', 'Broken curtain rail', 'LOW', 'RESOLVED', '2025-07-03', '2025-07-04'),
(15, 'R010', 'Shower pressure low', 'MEDIUM', 'OPEN', '2025-07-05', NULL);

SELECT * FROM maintenance;
```

Output:

57	15:26:13	INSERT INTO maintenance (maint_id, room_id, issue_desc, severity, status, reported_on, resolved...	15 row(s) affected Records: 15 Duplicates: 0 Warnings: 0
58	15:26:13	SELECT * FROM maintenance LIMIT 0, 50000	15 row(s) returned

[illegible]

V. INSERT INTO TABLE payments

The payments table contains 10 records capturing financial transactions made by students for rent and deposits. Each record includes the payment amount, date, method, and any relevant notes.

```
INSERT INTO payments (payment_id, amount, paid_on, method, note, student_id)
VALUES
```

```
(1, 550.00, '2025-01-15', 'CASH', 'January rent', 'A001'),
(2, 450.00, '2025-02-20', 'FPX', 'February rent', 'A002'),
(3, 950.00, '2025-03-10', 'CARD', 'March rent', 'A003'),
(4, 980.00, '2025-04-15', 'FPX', 'April rent', 'A004'),
(5, 820.00, '2025-05-12', 'CARD', 'May rent', 'A005'),
(6, 530.00, '2025-02-10', 'CASH', 'Deposit for new room', 'A006'),
(7, 460.00, '2025-03-05', 'TNG', 'Partial rent', 'A007'),
(8, 820.00, '2025-05-15', 'FPX', 'Full rent', 'A008'),
(9, 750.00, '2025-06-05', 'CARD', 'June rent', 'A009'),
(10, 950.00, '2025-06-10', 'TNG', 'June rent', 'A010');
```

```
SELECT * FROM payments;
```

Output:

	payment_id	student_id	amount	paid_on	method	note
▶	1	A001	550.00	2025-01-15	CASH	January rent
	2	A002	450.00	2025-02-20	FPX	February rent
	3	A003	950.00	2025-03-10	CARD	March rent
	4	A004	980.00	2025-04-15	FPX	April rent
	5	A005	820.00	2025-05-12	CARD	May rent
	6	A006	530.00	2025-02-10	CASH	Deposit for new room
	7	A007	460.00	2025-03-05	TNG	Partial rent
	8	A008	820.00	2025-05-15	FPX	Full rent
	9	A009	750.00	2025-06-05	CARD	June rent
	10	A010	950.00	2025-06-10	TNG	June rent
*	NULL	NULL	NULL	NULL	NULL	NULL

```
59 15:30:03 INSERT INTO payments (payment_id, amount, paid_on, method, note, student_id) VAL... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
60 15:30:03 SELECT * FROM payments LIMIT 0, 50000 10 row(s) returned
```

2. UPDATE and DELETE Operations

UPDATE is_occupied column to TRUE for rooms with active students; otherwise, set it to FALSE

```
UPDATE rooms
SET is_occupied = TRUE
WHERE room_id IN (
    SELECT room_id
    FROM students
    WHERE status = 'ACTIVE'
);

SET SQL_SAFE_UPDATES = 0;
DELETE FROM maintenance
WHERE status = 'RESOLVED' AND DATEDIFF(CURDATE(), reported_on) > 60;
```

Output:

61	15:42:05	UPDATE rooms SET is_occupied = TRUE WHERE room_id IN (SELECT room_id FROM ...	1 row(s) affected Rows matched: 7 Changed: 1 Warnings: 0
62	15:42:14	SET SQL_SAFE_UPDATES = 0	0 row(s) affected
63	15:42:14	DELETE FROM maintenance WHERE status = 'RESOLVED' AND DATEDIFF(CURDAT...	7 row(s) affected

[illegible]

3. Data Retrieval and Filtering Queries

Display room types where rent is between RM400 and RM800 using BETWEEN

```
SELECT *  
FROM room_types  
WHERE rent BETWEEN 400 AND 800;
```

Output:

66 15:51:23 SELECT * FROM room_types WHERE rent BETWEEN 400 AND 800 LIMIT 0, 50000 7 row(s) returned

	type_id	type_name	rent	deposit	capacity
▶	1	Single	550.00	250.00	1
	2	Double	450.00	150.00	2
	3	Family	750.00	350.00	4
	5	Single	530.00	200.00	1
	6	Double	420.00	120.00	2
	9	Single	580.00	220.00	1
	10	Double	460.00	180.00	2
*	NULL	NULL	NULL	NULL	NULL

Retrieve students whose first names (fname) start with the letter 'A' using LIKE

```
SELECT *  
FROM students  
WHERE fname LIKE 'A%';
```

Output:

67 15:54:14 SELECT * FROM students WHERE fname LIKE 'A%' LIMIT 0, 50000 1 row(s) returned

	student_id	room_id	fname	lname	status	checkin_date	email
▶	A001	R001	Aiman	Rahman	ACTIVE	2025-01-10	aiman@gmail.com
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Display payments where payment method is IN('FPX', 'CARD')

```
SELECT *  
FROM payments  
WHERE method IN ('FPX', 'CARD');
```

Output:

68 15:57:00 SELECT * FROM payments WHERE method IN ('FPX','CARD') LIMIT 0, 50000 6 row(s) returned

	payment_id	student_id	amount	paid_on	method	note
▶	2	A002	450.00	2025-02-20	FPX	February rent
	3	A003	950.00	2025-03-10	CARD	March rent
	4	A004	980.00	2025-04-15	FPX	April rent
	5	A005	820.00	2025-05-12	CARD	May rent
	8	A008	820.00	2025-05-15	FPX	Full rent
	9	A009	750.00	2025-06-05	CARD	June rent
	NULL	NULL	NULL	NULL	NULL	NULL

4. Functions and Expressions

Use COUNT() from TABLE payments

```
SELECT method, SUM(amount) AS total_payments
FROM payments
GROUP BY method;
```

Output:

69 16:03:00 SELECT method, SUM(amount) AS total_payments FROM payments GROUP BY metho... 4 row(s) returned

	method	total_payments
▶	CASH	1080.00
	FPX	2250.00
	CARD	2520.00
	TNG	1410.00

Use UPPER() and CONCAT() in your queries

```
SELECT student_id, UPPER(CONCAT(fname, ' ', lname)) AS student_full_name, email
FROM students;
```

Output:

69 16:03:00 SELECT method, SUM(amount) AS total_payments FROM payments GROUP BY metho... 4 row(s) returned

70 16:07:31 SELECT student_id, UPPER(CONCAT(fname, ' ', lname)) AS student_full_name, email F... 10 row(s) returned

	student_id	student_full_name	email
▶	A001	AIMAN RAHMAN	aiman@gmail.com
	A002	BELLA TAN	bella@gmail.com
	A003	CHONG WEI	chong@gmail.com
	A004	DINA AHMAD	dina@gmail.com
	A005	EDDY LOW	eddy@gmail.com
	A006	FARAH ALI	farah@gmail.com
	A007	GOH CHIN	goh@gmail.com
	A008	HANA ZULKIFLI	hana@gmail.com
	A009	IVAN LIM	ivan@gmail.com
	A010	JULIA TEO	julia@gmail.com

Task 3 – Reporting and Aggregation

1. Create a view v_room_status with columns

```
CREATE VIEW v_room_status AS
SELECT
    r.room_no,
    rt.type_name,
    rt.rent,
    r.floor_no,
    rt.capacity,
    (
        SELECT COUNT(*)
        FROM students s
        WHERE s.room_id = r.room_id
            AND s.status = 'ACTIVE'
    ) AS n_occupants,
    (
        SELECT COUNT(*)
        FROM maintenance m
        WHERE m.room_id = r.room_id
            AND m.status = 'OPEN'
    ) AS pending_issues,

    CASE
        WHEN (
            SELECT COUNT(*)
            FROM students s2
            WHERE s2.room_id = r.room_id
                AND s2.status = 'ACTIVE'
        ) = 0
        THEN TRUE ELSE FALSE
    END AS is_vacant

FROM rooms r
JOIN room_types rt ON r.type_id = rt.type_id;

SELECT * FROM v_room_status;
```

Output:

36 02:00:45 SELECT * FROM v_room_status LIMIT 0, 1000 10 row(s) returned

	room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
▶	A101	Single	550.00	1	1	1	0	0
	A102	Double	450.00	1	2	1	1	0
	A103	Family	750.00	1	4	0	2	1
	B201	Premium	950.00	2	2	1	0	0
	B202	Single	530.00	2	1	0	1	1
	B203	Double	420.00	3	2	1	1	0
	C301	Family	820.00	3	5	1	0	0
	C302	Premium	980.00	4	2	0	1	1
	C303	Single	580.00	4	1	1	0	0
	C304	Double	460.00	4	2	1	2	0

2. Summary queries (for the ≥ 10 -per-table dataset)

i. Total number of students per room type

```
# 1
SELECT rt.type_name, COUNT(s.student_id) AS total_students
FROM room_types rt
LEFT JOIN rooms r ON rt.type_id = r.type_id
LEFT JOIN students s ON r.room_id = s.room_id
GROUP BY rt.type_name;
```

Output:

37 02:07:45 SELECT rt.type_name, COUNT(s.student_id) AS total_students FROM room_types rt LEFT JOIN rooms r ON rt.... 4 row(s) returned

	type_name	total_students
▶	Single	2
	Double	3
	Family	1
	Premium	1

ii. Average rent and total deposit per room type

```
SELECT
    type_name,
    ROUND(AVG(rent), 2) AS avg_rent,
    ROUND(SUM(deposit), 2) AS total_deposit
FROM room_types
GROUP BY type_name;
```


Output:

```
38 02:10:06 SELECT type_name, ROUND(AVG(rent), 2) AS avg_rent, ROUND(SUM(deposit), 2) AS total_deposit ... 4 row(s) returned
```

	type_name	avg_rent	total_deposit
▶	Single	553.33	670.00
	Double	443.33	450.00
	Family	785.00	650.00
	Premium	965.00	950.00

iii. Monthly payment totals grouped by year & month

```
SELECT
    YEAR(paid_on) AS pay_year,
    MONTH(paid_on) AS pay_month,
    SUM(amount) AS total_amount
FROM payments
GROUP BY YEAR(paid_on), MONTH(paid_on)
ORDER BY pay_year, pay_month;
```

Output:

```
39 02:13:04 SELECT YEAR(paid_on) AS pay_year, MONTH(paid_on) AS pay_month, SUM(amount) AS total_amount... 6 row(s) returned
```

	pay_year	pay_month	total_amount
▶	2025	1	550.00
	2025	2	980.00
	2025	3	1410.00
	2025	4	980.00
	2025	5	1640.00
	2025	6	1700.00

iv. Count of OPEN maintenance issues per floor using HAVING COUNT > 2

```
SELECT
    r.floor_no,
    COUNT(m.maint_id) AS open_issues
FROM maintenance m
JOIN rooms r ON r.room_id = m.room_id
WHERE m.status = 'OPEN'
GROUP BY r.floor_no
HAVING COUNT(m.maint_id) > 2;
```

Output:

✓ 40 02:14:58 SELECT r.floor_no, COUNT(m.maint_id) AS open_issues FROM maintenance m JOIN rooms r ON r.room_id ... 2 row(s) returned

	floor_no	open_issues
▶	1	3
	4	3

3. Apply SQL functions in your report queries:

```
SELECT
    r.room_no,
    UPPER(CONCAT(s.fname, ' ', s.lname)) AS student_name,
    rt.type_name,
    ROUND(rt.rent, 2) AS rent_rounded,
    r.floor_no,
    rt.capacity,
    CASE
        WHEN rt.rent < 500 THEN 'LOW'
        WHEN rt.rent >= 500 AND rt.rent < 800 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS rent_category
FROM rooms r
JOIN room_types rt ON r.type_id = rt.type_id
JOIN students s ON s.room_id = r.room_id
ORDER BY r.room_no;
```

Output:

✓ 52 02:34:55 SELECT r.room_no, UPPER(CONCAT(s.fname, ' ', s.lname)) AS student_name, rt.type_name, ROU... 7 row(s) returned

	room_no	student_name	type_name	rent_rounded	floor_no	capacity	rent_category
▶	A101	AIMAN RAHMAN	Single	550.00	1	1	MEDIUM
	A102	BELLA TAN	Double	450.00	1	2	LOW
	B201	CHONG WEI	Premium	950.00	2	2	HIGH
	B203	DINA AHMAD	Double	420.00	3	2	LOW
	C301	EDDY LOW	Family	820.00	3	5	HIGH
	C303	IVAN LIM	Single	580.00	4	1	MEDIUM
	C304	JULIA TEO	Double	460.00	4	2	LOW

MARKING RUBRICS

Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (PK, FK, NOT NULL, UNIQUE). Logical order and referential integrity preserved.	3	
	ALTER TABLE (add email UNIQUE) and DROP TABLE demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data UPDATE/DELETE executed correctly with logical results.	1	
	Accurate use of WHERE, BETWEEN, LIKE, IN, AND/OR/NOT queries.	3	
	Correct use of aggregate and string functions (COUNT, CONCAT, and more) .	2	
Task 3 Reporting & Aggregation	VIEW creation	2	
	Use of GROUP BY, HAVING, ROUND, and CASE with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
Total (20)			